Programming Assignment: Tomasulo Algorithm

Mostafa Mushsharat
ID: 2400966

1 Introduction

Tomasulo Algorithm. Tomasulo's algorithm is used for dynamic scheduling of instructions. It allows out-of-order execution through register renaming and reservation stations, and combined with a reorder buffer (ROB) it keeps the architectural state in program order, which makes speculative execution past branches possible.

In this assignment I have done the following:

1. Fixed and completed the existing Python implementation of the Tomasulo algorithm (the EX, MEM, WB and COMMIT stages had several timing and bookkeeping bugs, see Section 2).
2. Added support for issuing, writing back and committing multiple instructions per cycle (issue width 1–4, in order).
3. Added the branch unit described in the assignment: a one-bit predictor with an 8-entry branch target buffer, speculative fetch past branches, and the recovery actions on a misprediction (RAT recovery, clearing the reservation stations and the ROB entries that surpass the branch, fetch restarts at cycle n+2).
4. Made the hardware configuration, the issue width, the initial register/memory values and the program all come from the input file, and added the register and memory dump required by the assignment's output format.
5. Tested the modification with the provided test case and with my own test cases at every issue width.

What Works

- Multiple issue, multiple write-back and multiple commit (issue width 1 to 4), all in program order.
- The FP adder and FP multiplier are pipelined, the integer adder is not; the memory is single ported and not pipelined. More than one FU per type is supported for the ALU units (`# of FUs` column).
- Load/store queue with effective address calculation in EX, memory disambiguation (a load waits until the addresses of all earlier stores are known), store-to-load forwarding (one cycle in MEM) and stores that write the memory only when they commit.
- `Bne` and `Beq` with the one-bit predictor + BTB, speculative execution and full recovery after a misprediction. Wrong-path instructions never commit and never change the architectural state.
- `R0` is hardwired to 0. The input text is case insensitive and commas are optional.
- The final registers and memory are identical for every issue width on every test case (only the timing changes), which is the main correctness check for the multi-issue

support.

Limitations

- The `# of FUs` column of the load/store unit is parsed but only one load/store unit (one address adder and one memory port) is modelled, because the assignment specifies a single-ported, non-pipelined memory.
- The number of CDB broadcasts per cycle is equal to the issue width (the same choice as in the sample report), otherwise a 4-wide machine would be throttled by a 1-wide CDB.
- A fetch group ends at a branch: the instructions after a branch are fetched from the predicted PC in the next cycle, even when the prediction is "not taken".
- The branch offset is interpreted as a byte offset relative to PC+4 (`target = PC + 1 + offset/4` in words), which is what the original code did and what the provided test case (`Bne R1 R0 -12`, three iterations over the two `Addi`) requires. With the literal `PC+4+offset<<2` formula that test would jump to a negative address.
- Squashed (wrong path) instructions are not printed in the status table, only committed instructions are.

2 Code Changes

In this section I discuss all the changes I made to the existing Tomasulo implementation (the Python version), file by file. The general timing convention used everywhere is: a stage can only use something that was produced in an **earlier** cycle. An instruction issued in cycle N starts EX in N+1 at the earliest, a value broadcast on the CDB in cycle N is usable by EX in N+1, a result produced in cycle N is broadcast in N+1, and an instruction commits at least one cycle after its WB. A non-pipelined unit that finishes in cycle N accepts a new instruction in N+1. The stage functions are still called in the original order (`issue`, `exe`, `mem`, `wb`, `commit`); instead of reordering them I made every stage check the cycle in which its input became ready.

2.1 `main.py`

(1) Configuration from the input file. The original `main.py` hard-coded the register values, the memory contents and the sizes/latencies of all units and read the program from `code.in`. Now everything is read from one input file (`python main.py <file>`), as the assignment requires, including the issue width:

Initial Version

```
        instructions = read_instruction('code.in')
        ...
        reg_int[1] = 12
        reg_int[2] = 32
        ...
        size_rs_int_adder = 4
        rs_int_adder = build_rs(size_rs_int_adder)
```

My Version

```
        if len(sys.argv) > 1:
            codefile = sys.argv[1]
        else:
            codefile = 'code.in'
        config, instructions = read_input(codefile)
        issue_width = config['issue_width']
        ...
        for i in config['reg_int']:
            reg_int[i] = config['reg_int'][i]
        reg_int[0] = 0
        ...
        rs_int_adder = build_rs(config['rs_int_adder'])
```

(2) Branch target buffer. An 8-entry BTB is built (`btb = build_btb(8)`) and passed to `issue` (prediction) and `exe` (resolution/update).

(3) The main loop. The loop condition was `(len(ROB)>0)|(cycle==1)`. After a misprediction the ROB can be empty while fetch is paused for the recovery cycle, so the loop would have stopped too early. It is now `while (len(ROB)>0) or (PC.PC<len(instructions))`. At the top of every cycle fetch is re-enabled when the recovery cycle has passed:

```
        if (PC.valid==0) and (cycle>=PC.resume):
            PC.valid = 1
```

After the loop a store that is still writing the memory is finished (`while ld_sd_mem.busy == 1: mem(...)`).

(4) Output. After the instruction status table, `print_registers` and `print_memory` print the register values and the non-zero memory values in the format of the assignment. The CDB latch (`cdb` object) is no longer created because the WB stage now broadcasts directly from the results queue (see 2.6).

2.2 `init.py`

(1) `read_input` replaces `read_instruction`. It parses the configuration lines (`Integer adder 4 1 1`, `Load/store unit 5 1 5 1`, `ROB entries = 64`, `Issue width = 2`), the initial values (`R1=12, F20=3.0`, `Mem[4]=3.0`) and the instructions. Values with a decimal point become floats, the others integers, so a value loaded into an integer register can be used as a base address. `normalize_instruction` makes the input case insensitive and comma tolerant by rewriting every instruction into the canonical form the rest of the code expects (`Add.d F1 F2 F3`), so the original `split(' ')` parsing in the other files did not have to change.

(2) New fields. `fu_result` got a `cycle` field (the cycle in which a result was produced) so that the WB stage can broadcast it only in a later cycle. `ROB_entry` got a `predict` field (the next PC predicted when a branch was issued). `PC` got `resume` (the cycle in which fetch restarts after a misprediction) and `squash` (the ROB tag of the mispredicted branch that still has to be handled). `ld_sd_exe` and `ld_sd_mem` got an `end` field (the last cycle they were busy) so a non-pipelined unit cannot be reused in the cycle it finishes. `ld_sd_entry` initialises `data/address` to `None` (before, reading them on a load returned a namedtuple descriptor, and the code only worked by accident).

(3) BTB. `btb_entry (pc, target, taken)` and `build_btb(num)` implement the 8-entry buffer with the one-bit predictor.

2.3 `issue.py`

(1) Issue width. The old `issue` issued exactly one instruction. It is now a loop around a new `issue_one` function; it stops at the first instruction that cannot be issued (ROB, reservation station or load/store queue full) so that issue stays in order, and it stops after a branch (fetch continues from the predicted PC in the next cycle).

Initial Version

```
        def issue(cycle, PC, instructions, ROB, size_ROB, ... rat_int, rat_fp):
            # fetch 1 instruction
            ins = instructions[PC.PC]
            ...
```

My Version

```
        def issue(cycle, PC, instructions, ROB, size_ROB, ... rat_int, rat_fp, btb,
issue_width):
            for i in range(issue_width):
                if PC.PC >= len(instructions):
                    break
                ins = instructions[PC.PC]
                issued = issue_one(cycle, PC, instructions, ROB, size_ROB, ... rat_int,
rat_fp, btb)
                if issued == False:
                    break
                # a branch ends the fetch group of this cycle
                if (ins.split(' ')[0]=='Bne') or (ins.split(' ')[0]=='Beq'):
                    break
```

(2) Branch prediction instead of stalling. The original code stopped fetching after a branch (`PC.valid = 0`) until the branch was resolved, so nothing could ever surpass a branch. Now the branch is predicted at issue with the BTB (`predict_branch`): if the BTB entry for `PC & 7` belongs to this PC and its bit says taken, fetch continues from the stored target, otherwise from PC+1. The prediction is remembered in the ROB entry so the EX stage can detect a misprediction:

```
        ROB[-1].predict = predict_branch(btb, PC)
        PC.PC = ROB[-1].predict
```

`Beq` is accepted everywhere `Bne` was.

(3) Operand reading. The register-reading code was repeated six times (`put_ins_into_rs`, `put_bne_into_rs`, `put_ins_into_ldsd`); it is now one helper `read_register` returning `(tag, value, valid)`. The `offset(Rx)` operand of loads/stores was parsed with fixed string positions (`[:-4]`, `[-3:-1]`), which only worked for one-digit register numbers; it now uses the position of the parentheses. `find_tag_new_ldsd_entry` picked a free queue tag out of a `set` (random order); it now takes the lowest free one.

(4) R0. `update_rat` never renames `R0`, so reads of `R0` always see 0, and `commit.py` never writes it.

2.4 `exe.py`

(1) Functional unit timing. This was the main bug of the original EX stage. `fu_exe` advanced every instruction in a unit and then, when it dispatched a new instruction, called `fu_exe` again, which advanced all the other instructions a second time in the same cycle (the sample report worked around this with a `cycle_call_history` dictionary). I split the function into `fu_exe`

(advance everything already in the unit by one cycle, then finish what is done) and `fu_dispatch` (move ready instructions from the reservation station into the unit). A dispatched instruction starts with `cycle = 1`, its start cycle is recorded, and one-cycle units finish immediately through the shared `fu_finish`:

Initial Version

```
        def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
            if len(fu)!=0:
                for element in fu:
                    element.cycle += 1
                if fu[-1].cycle == 1:            # start cycle of the newest entry
                    ...
                if fu[0].cycle == time_fu:       # finish cycle of the oldest entry
                    ...
                    fu.popleft()
        ...
                fu_int_adder.append(fu_entry())
                add_entry_into_fu(fu_int_adder, rs_int_adder[index])
                if ROB[...].issue[0] < cycle:
                    fu_exe(fu_int_adder, ...)    # advances everybody again!
```

My Version

```
        def fu_finish(fu, fu_results, ROB, time_fu, cycle, PC, btb):
            while (len(fu) > 0) and (fu[0].cycle >= time_fu):
                entry = fu.popleft()
                index = find_ROB_entry(ROB, entry.dest_tag)
                ROB[index].exe.append(cycle)
                if (entry.op=='Bne') or (entry.op=='Beq'):
                    if PC.squash == '':
                        resolve_branch(entry, ROB[index], cycle, PC, btb)
                else:
                    fu_results.append(fu_result())      # value, dest_tag and cycle
                    ...

        def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC, btb):
            for element in fu:
                element.cycle += 1
            fu_finish(fu, fu_results, ROB, time_fu, cycle, PC, btb)

        def fu_dispatch(rs, fu, fu_results, ROB, time_fu, num_fu, pipelined, in_use,
cycle, PC, btb):
            for n in range(num_fu):
                if (pipelined == False) and (in_use + n >= num_fu):
                    break
                index = check_valid_ins_in_rs(rs, ROB, cycle)
                if index < 0:
                    break
                add_entry_into_fu(fu, rs[index])
                ROB[find_ROB_entry(ROB, rs[index].dest_tag)].exe.append(cycle)
                rs[index].busy = 0
                fu_finish(fu, fu_results, ROB, time_fu, cycle, PC, btb)
```

`num_fu` comes from the `# of FUs` column: a pipelined unit can accept `num_fu` instructions per cycle, the unpipelined integer adder can hold at most `num_fu` instructions (`in_use` is the number it held at the start of the cycle, so an instruction finishing in cycle N does not free the adder for cycle N).

(2) Oldest ready instruction first. `check_valid_ins_in_rs` returned the first ready entry in reservation-station order, which is not program order. It now returns the ready entry with the

lowest ROB position that was issued in an earlier cycle (the sample report did the same with issue cycles). `check_valid_ins_in_ldsd` got the same "issued in an earlier cycle" check. `find_ROB_entry` returns -1 instead of the last index when a tag is not found.

(3) Address calculation. `ld_sd_execution` is split in the same way (`ld_sd_execution` advances, `ld_sd_finish` writes the address back to the queue and records the end cycle). The address is converted to `int`. The address adder is reused only in the cycle after it finishes (`ld_sd_exe.end < cycle`).

(4) Branch resolution. `resolve_branch` computes the outcome (`!=` for `Bne`, `==` for `Beq`) and the target, updates the BTB entry (target and one-bit prediction = last outcome), and compares the real next PC with the predicted one stored in the ROB entry. On a misprediction it sets the correct PC, stops fetch, and schedules the restart and the squash:

```
        if next_pc != rob_entry.predict:
            PC.PC = next_pc
            PC.valid = 0
            PC.resume = cycle + 2     # recovery takes one cycle, fetch restarts at n+2
            PC.squash = rob_entry.ROB_tag
```

The original code computed `PC.PC + 1 + offset/4` with the *current* fetch PC, which was only right because fetch had been stalled at the branch; it now uses the PC stored in the branch's ROB entry.

(5) Squashing. `squash` is called at the end of `exe` in the cycle the misprediction is detected. It performs the three actions listed in the assignment: (a) it pops every ROB entry behind the branch (`while ROB[-1].ROB_tag != branch_tag: ROB.pop()`), (b) it clears the reservation station entries, the entries inside the functional units, the results waiting for the CDB, the load/store queue entries, the address adder and the memory unit that belong to the flushed ROB tags, and (c) it recovers the RAT with `recover_rat`: the RAT is rebuilt from the architectural registers and the renaming is redone for the instructions that are still in the ROB (tag if not yet broadcast, value otherwise):

```
        def recover_rat(ROB, rat_int, rat_fp, reg_int, reg_fp):
            for i in range(32):
                rat_int[i] = reg_int[i]
                rat_fp[i] = reg_fp[i]
            for entry in ROB:
                ...
                if len(entry.cdb) == 0:
                    rat[int(entry.dest_tag[1:])] = entry.ROB_tag
                else:
                    rat[int(entry.dest_tag[1:])] = entry.value
```

2.5 `mem.py`

(1) Dead and wrong code removed. `check_if_sd_committable` referenced a field that does not exist (`ldsd_tag`) and was never actually called (`if (check_if_sd_committable):` tested the function object, which is always true). Stores no longer go through `mem.py` at all: they are written at commit (see 2.7), as the assignment requires ("stores write into the memory only in the commit stage").

(2) Timing. A load can go to the memory (or forward) only if its address was computed in an earlier cycle (`address_ready`, which looks at the EX end cycle in the ROB); the same check is used for the addresses of the earlier stores. The memory unit counts its own cycles and is free

again only in the cycle after it finishes (`memory_free` uses the `end` field). The original code started the memory access in the same cycle the previous access finished, and started counting one cycle late, which is why a 5-cycle access showed up as `[3, 7]` but its result only appeared at cycle 9.

(3) Forwarding and disambiguation. `forward_check_from_sd` now walks back to the nearest earlier store *with the same address* (skipping stores with other addresses) and returns its data when it is known; if an earlier store address is still unknown the load waits. Forwarding takes one cycle in MEM (`mem = [N]`, result produced in N and broadcast in N+1). `check_all_previous_Sd_no_match` is used, as before, to decide that a load may go to the memory.

(4) Priority. Loads go to the memory oldest first. A ready store at the head of the ROB is older than every load in the queue, so `head_sd_waiting` keeps younger loads from taking the single memory port in the cycle the store commits.

2.6 `wb.py`

(1) Multiple broadcasts. The original WB used a one-entry latch (`cdb`): it fetched a result into the latch in cycle N and broadcast it in N+1, strictly in the order results arrived. Now `wb` broadcasts up to `issue_width` results per cycle straight from `results_buffer`, choosing each time the **oldest** result (lowest ROB position) that was produced in an earlier cycle (`result.cycle < cycle`). This keeps the "result at N, WB at N+1" timing and gives the CDB to the older instruction when two finish in the same cycle (in the original an `Addi` could steal the bus from an older `Ld`).

Initial Version

```
        if cdb.valid == 1:
            boroadcast(cdb, ...)
            cdb.valid = 0
        if cdb.valid == 0:
            if len(results_buffer)>0:
                cdb.valid = 1
                cdb.value = results_buffer[0].value
                ...
                results_buffer.popleft()
```

My Version

```
        count = 0
        while count < issue_width:
            pick = None
            oldest = len(ROB)
            for result in results_buffer:
                if result.cycle < cycle:
                    index = find_ROB_entry(ROB, result.dest_tag)
                    if index < oldest:
                        oldest = index
                        pick = result
            if pick == None:
                break
            results_buffer.remove(pick)
            boroadcast(pick, ...)
            count += 1
```

(2) RAT bug. `boroadcast` wrote the value into the RAT unconditionally. If a younger instruction had already renamed the same register to a newer ROB tag, the broadcast overwrote that tag

with a stale value and later readers got the wrong operand. The RAT is now only updated when it still points at the broadcasting ROB tag. Reservation station entries that are not busy are skipped, and the ROB entry is looked up once instead of scanning the whole ROB.

2.7 `commit.py`

(1) Multiple commit. `commit` is now a loop over `issue_width`; it stops at the first ROB head that is not finished, so commit stays in order. An instruction commits only if it was broadcast in an earlier cycle (`head.cdb[0] < cycle`); before, the commit cycle was written as `cycle+1`, i.e. the entry was removed one cycle early and the printed number was fudged.

(2) Branches. The original popped a `Bne` as soon as it had an EX cycle and never recorded a commit cycle (the COMMIT column of branches was empty). A branch now commits in order, one cycle after it was resolved, and its commit cycle is printed. The WB column of a branch stays empty because a branch produces no result.

(3) Stores. A `Sd` commits when it is at the ROB head, its address and data are known and the memory is free; at that moment it leaves the load/store queue and writes the memory, which occupies the memory unit for `Cycles in Mem` cycles (shown in the MEM column, e.g. `[31, 35]` with COMMIT `[31]`). The original code sent the store to the memory from `mem.py` and wrote its commit cycle there, which could happen before older instructions had committed.

(4) R0. `modify_arch_reg` never writes `R0`.

2.8 `print_status.py`

Added `print_registers` (R0–R31 and F0–F31, eight per row) and `print_memory` (non-zero byte addresses and values).

2.9 Differences from the sample report

With the sample report's test case #01 my simulator produces the same table except for three places, all of which come from deliberate modelling choices: (a) when two instructions finish in the same cycle the older one gets the CDB (the sample gave it to the load), (b) a reservation station freed in cycle N is reused by an instruction issued in cycle N+1 (the sample reused it in cycle N by calling `exe` before `issue`), and (c) an `Addi` waits for the CDB broadcast of its operand like every other instruction (the sample special-cased `Addi` and let it execute in the broadcast cycle). The comparison is in Section 3.4.

3 Results

All outputs below were produced with `run_tests.sh`, which runs every file in `testcases/` with issue width 1, 2, 3 and 4 (the complete outputs, including the register and memory dumps, are in `results/`). For every test case the final register and memory contents are identical for all four issue widths.

3.1 Test Case #01 (provided test case)

Configuration of the Architecture

```
               # of rs         Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4               1                               1
FP adder        3               4                               1
FP multiplier   2               15                              1
Load/store unit 5               1               5               1
ROB entries = 64
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0
```

List of Instructions

```
Ld F2 0(R1)
Mult.d F4 F2 F20
Ld F6 0(R2)
Add.d F6 F4 F6
Sd F6 0(R2)
Addi R1 R1 -4
Addi R2 R2 -4
Bne R1 R0 -12
Add.d F20 F2 F2
```

The branch jumps back to the first `Addi` (byte offset -12 from PC+4), so the two `Addi` and the `Bne` run three times (R1 = 12, 8, 4, 0). The first `Bne` is predicted not taken (empty BTB) and is taken: the speculatively issued `Add.d F20 F2 F2` is squashed and fetch restarts two cycles after the branch was resolved. The BTB then predicts taken for the second and third `Bne`; the third one is not taken, so the wrongly fetched `Addi` instructions are squashed and `Add.d F20 F2 F2` is fetched again at n+2.

Issue width 1

```
                   ISSUE     EXE         MEM         WB        COMMIT
Ld F2 0(R1)        [1]       [2, 2]      [3, 7]      [8]       [9]
Mult.d F4 F2 F20   [2]       [9, 23]     []          [24]      [25]
Ld F6 0(R2)        [3]       [4, 4]      [8, 12]     [13]      [26]
Add.d F6 F4 F6     [4]       [25, 28]    []          [29]      [30]
Sd F6 0(R2)        [5]       [6, 6]      [31, 35]    []        [31]
Addi R1 R1 -4      [6]       [7, 7]      []          [9]       [32]
Addi R2 R2 -4      [7]       [8, 8]      []          [10]      [33]
Bne R1 R0 -12      [8]       [10, 10]    []          []        [34]
Addi R1 R1 -4      [12]      [13, 13]    []          [14]      [35]
Addi R2 R2 -4      [13]      [14, 14]    []          [15]      [36]
Bne R1 R0 -12      [14]      [15, 15]    []          []        [37]
Addi R1 R1 -4      [15]      [16, 16]    []          [17]      [38]
Addi R2 R2 -4      [16]      [17, 17]    []          [18]      [39]
Bne R1 R0 -12      [17]      [18, 18]    []          []        [40]
Add.d F20 F2 F2    [20]      [21, 24]    []          [25]      [41]
```

Issue width 2

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6 0(R2)	[2]	[3, 3]	[8, 12]	[13]	[25]
Add.d F6 F4 F6	[2]	[25, 28]	[]	[29]	[30]
Sd F6 0(R2)	[3]	[4, 4]	[30, 34]	[]	[30]
Addi R1 R1 -4	[3]	[4, 4]	[]	[5]	[31]
Addi R2 R2 -4	[4]	[5, 5]	[]	[6]	[31]
Bne R1 R0 -12	[4]	[6, 6]	[]	[]	[32]
Addi R1 R1 -4	[8]	[9, 9]	[]	[10]	[32]
Addi R2 R2 -4	[8]	[10, 10]	[]	[11]	[33]
Bne R1 R0 -12	[9]	[11, 11]	[]	[]	[33]
Addi R1 R1 -4	[10]	[12, 12]	[]	[13]	[34]
Addi R2 R2 -4	[10]	[13, 13]	[]	[14]	[34]
Bne R1 R0 -12	[11]	[14, 14]	[]	[]	[35]
Add.d F20 F2 F2	[16]	[17, 20]	[]	[21]	[35]

Issue width 3

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6 0(R2)	[1]	[3, 3]	[8, 12]	[13]	[25]
Add.d F6 F4 F6	[2]	[25, 28]	[]	[29]	[30]
Sd F6 0(R2)	[2]	[4, 4]	[30, 34]	[]	[30]
Addi R1 R1 -4	[2]	[3, 3]	[]	[4]	[30]
Addi R2 R2 -4	[3]	[4, 4]	[]	[5]	[31]
Bne R1 R0 -12	[3]	[5, 5]	[]	[]	[31]
Addi R1 R1 -4	[7]	[8, 8]	[]	[9]	[31]
Addi R2 R2 -4	[7]	[9, 9]	[]	[10]	[32]
Bne R1 R0 -12	[7]	[10, 10]	[]	[]	[32]
Addi R1 R1 -4	[8]	[11, 11]	[]	[12]	[32]
Addi R2 R2 -4	[9]	[12, 12]	[]	[13]	[33]
Bne R1 R0 -12	[10]	[13, 13]	[]	[]	[33]
Add.d F20 F2 F2	[15]	[16, 19]	[]	[20]	[33]

Issue width 4

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6 0(R2)	[1]	[3, 3]	[8, 12]	[13]	[25]
Add.d F6 F4 F6	[1]	[25, 28]	[]	[29]	[30]
Sd F6 0(R2)	[2]	[4, 4]	[30, 34]	[]	[30]
Addi R1 R1 -4	[2]	[3, 3]	[]	[4]	[30]
Addi R2 R2 -4	[2]	[4, 4]	[]	[5]	[30]
Bne R1 R0 -12	[2]	[5, 5]	[]	[]	[31]
Addi R1 R1 -4	[7]	[8, 8]	[]	[9]	[31]
Addi R2 R2 -4	[7]	[9, 9]	[]	[10]	[31]
Bne R1 R0 -12	[7]	[10, 10]	[]	[]	[31]
Addi R1 R1 -4	[8]	[11, 11]	[]	[12]	[32]
Addi R2 R2 -4	[9]	[12, 12]	[]	[13]	[32]
Bne R1 R0 -12	[10]	[13, 13]	[]	[]	[32]
Add.d F20 F2 F2	[15]	[16, 19]	[]	[20]	[32]

Final state (identical for all widths)

```
REGISTER VALUES

        R0      R1      R2      R3      R4      R5      R6      R7
value:  0       0       20      0       0       0       0       0
        R8      R9      R10     R11     R12     R13     R14     R15
value:  0       0       0       0       0       0       0       0
        R16     R17     R18     R19     R20     R21     R22     R23
value:  0       0       0       0       0       0       0       0
        R24     R25     R26     R27     R28     R29     R30     R31
value:  0       0       0       0       0       0       0       0
        F0      F1      F2      F3      F4      F5      F6      F7
value:  0.0     0.0     1.0     0.0     3.0     0.0     7.0     0.0
        F8      F9      F10     F11     F12     F13     F14     F15
value:  0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0
        F16     F17     F18     F19     F20     F21     F22     F23
value:  0.0     0.0     0.0     0.0     2.0     0.0     0.0     0.0
        F24     F25     F26     F27     F28     F29     F30     F31
value:  0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0

NON-ZERO MEMORY VALUES

Addresses      Values
4              3.0
8              2.0
12             1.0
24             6.0
28             5.0
32             7.0
```

3.2 Test Case #02 (loop over the whole body)

Same configuration and initial values as test case #01, but the branch offset is -32 so the loop body is the whole program: three iterations load `Mem[12]`, `Mem[8]`, `Mem[4]` and `Mem[32]`, `Mem[28]`, `Mem[24]` and store `F4 + F6` back. This exercises stores and loads inside a mispredicted/predicted loop, the memory disambiguation (the loads of the next iteration go to different addresses than the pending store) and the ROB/RS clean-up after squashing with FP instructions in flight.

List of Instructions

```
Ld F2 0(R1)
Mult.d F4 F2 F20
Ld F6 0(R2)
Add.d F6 F4 F6
Sd F6 0(R2)
Addi R1 R1 -4
Addi R2 R2 -4
Bne R1 R0 -32
Add.d F20 F2 F2
```

Issue width 1

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4 F2 F20	[2]	[9, 23]	[]	[24]	[25]
Ld F6 0(R2)	[3]	[4, 4]	[8, 12]	[13]	[26]
Add.d F6 F4 F6	[4]	[25, 28]	[]	[29]	[30]
Sd F6 0(R2)	[5]	[6, 6]	[34, 38]	[]	[34]
Addi R1 R1 -4	[6]	[7, 7]	[]	[9]	[35]
Addi R2 R2 -4	[7]	[8, 8]	[]	[10]	[36]
Bne R1 R0 -32	[8]	[10, 10]	[]	[]	[37]
Ld F2 0(R1)	[12]	[13, 13]	[14, 18]	[19]	[38]
Mult.d F4 F2 F20	[13]	[20, 34]	[]	[35]	[39]
Ld F6 0(R2)	[14]	[15, 15]	[19, 23]	[25]	[40]
Add.d F6 F4 F6	[15]	[36, 39]	[]	[40]	[41]
Sd F6 0(R2)	[16]	[17, 17]	[42, 46]	[]	[42]
Addi R1 R1 -4	[17]	[18, 18]	[]	[20]	[43]
Addi R2 R2 -4	[18]	[19, 19]	[]	[21]	[44]
Bne R1 R0 -32	[19]	[21, 21]	[]	[]	[45]
Ld F2 0(R1)	[20]	[21, 21]	[24, 28]	[30]	[46]
Mult.d F4 F2 F20	[21]	[31, 45]	[]	[46]	[47]
Ld F6 0(R2)	[22]	[23, 23]	[29, 33]	[34]	[48]
Add.d F6 F4 F6	[23]	[47, 50]	[]	[51]	[52]
Sd F6 0(R2)	[24]	[25, 25]	[53, 57]	[]	[53]
Addi R1 R1 -4	[25]	[26, 26]	[]	[27]	[54]
Addi R2 R2 -4	[26]	[27, 27]	[]	[28]	[55]
Bne R1 R0 -32	[27]	[28, 28]	[]	[]	[56]
Add.d F20 F2 F2	[30]	[31, 34]	[]	[36]	[57]

Issue width 4

	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6 0(R2)	[1]	[3, 3]	[8, 12]	[13]	[25]
Add.d F6 F4 F6	[1]	[25, 28]	[]	[29]	[30]
Sd F6 0(R2)	[2]	[4, 4]	[33, 37]	[]	[33]
Addi R1 R1 -4	[2]	[3, 3]	[]	[4]	[33]
Addi R2 R2 -4	[2]	[4, 4]	[]	[5]	[33]
Bne R1 R0 -32	[2]	[5, 5]	[]	[]	[33]
Ld F2 0(R1)	[7]	[8, 8]	[13, 17]	[18]	[34]
Mult.d F4 F2 F20	[7]	[19, 33]	[]	[34]	[35]
Ld F6 0(R2)	[7]	[9, 9]	[18, 22]	[23]	[35]
Add.d F6 F4 F6	[7]	[35, 38]	[]	[39]	[40]
Sd F6 0(R2)	[8]	[10, 10]	[40, 44]	[]	[40]
Addi R1 R1 -4	[8]	[9, 9]	[]	[10]	[40]
Addi R2 R2 -4	[8]	[10, 10]	[]	[11]	[40]
Bne R1 R0 -32	[8]	[11, 11]	[]	[]	[41]
Ld F2 0(R1)	[9]	[11, 11]	[23, 27]	[28]	[41]
Mult.d F4 F2 F20	[10]	[29, 43]	[]	[44]	[45]
Ld F6 0(R2)	[14]	[15, 15]	[28, 32]	[33]	[45]
Add.d F6 F4 F6	[14]	[45, 48]	[]	[49]	[50]
Sd F6 0(R2)	[19]	[20, 20]	[50, 54]	[]	[50]
Addi R1 R1 -4	[19]	[20, 20]	[]	[21]	[50]
Addi R2 R2 -4	[19]	[21, 21]	[]	[22]	[50]
Bne R1 R0 -32	[19]	[22, 22]	[]	[]	[51]
Add.d F20 F2 F2	[26]	[29, 32]	[]	[33]	[51]

Final state (identical for all widths): `R1 = 0`, `R2 = 20`, `F20 = 6.0`, and

```
Addresses      Values
4              3.0
8              2.0
12             1.0
24             15.0
28             11.0
32             7.0
```

Note the store at the head of the ROB waiting for the memory port (e.g. `Sd F6 0(R2)` committing at 34 with issue width 1): the loads of the following iterations were allowed to go to the memory early, and the single-ported memory is busy when the store becomes ready to commit.

3.3 Test Case #03 (forwarding, Beq, R0)

Configuration of the Architecture

```
               # of rs         Cycles in EX    Cycles in Mem   # of FUs
Integer adder   2               1                               1
FP adder        3               3                               1
FP multiplier   2               20                              1
Load/store unit 3               1               4                               1
ROB entries = 128
R1=12, R2=12, F0=1.5
Mem[12]=2.0, Mem[20]=10.0
```

List of Instructions

```
Ld F1, 0(R1)
Add.d F2, F1, F0
Sd F2, 8(R1)
Ld F3, 8(R1)
Sub.d F4, F3, F1
Beq R1, R2, 8
Add R3, R1, R2
Sub R4, R3, R1
Mult.d F5, F4, F2
Add R0, R1, R2
```

`Ld F3 8(R1)` has the same address as the preceding `Sd F2 8(R1)`; it waits for the store data and then gets it by forwarding in one MEM cycle. `Beq R1 R2 8` is taken (R1 == R2) but predicted not taken, so `Add R3` and `Sub R4` are squashed (R3 = R4 = 0 at the end). `Add R0 R1 R2` executes but R0 stays 0.

Issue width 1

```
                   ISSUE     EXE         MEM         WB        COMMIT
Ld F1 0(R1)        [1]       [2, 2]      [3, 6]      [7]       [8]
Add.d F2 F1 F0     [2]       [8, 10]     []          [11]      [12]
Sd F2 8(R1)        [3]       [4, 4]      [13, 16]    []        [13]
Ld F3 8(R1)        [4]       [5, 5]      [12]        [13]      [14]
Sub.d F4 F3 F1     [5]       [14, 16]    []          [17]      [18]
Beq R1 R2 8        [6]       [7, 7]      []          []        [19]
Mult.d F5 F4 F2    [9]       [18, 37]    []          [38]      [39]
Add R0 R1 R2       [10]      [11, 11]    []          [12]      [40]
```

Issue width 4

```
                    ISSUE     EXE         MEM         WB        COMMIT
Ld F1 0(R1)         [1]       [2, 2]      [3, 6]      [7]       [8]
Add.d F2 F1 F0      [1]       [8, 10]     []          [11]      [12]
Sd F2 8(R1)         [1]       [3, 3]      [12, 15]    []        [12]
Ld F3 8(R1)         [1]       [4, 4]      [12]        [13]      [14]
Sub.d F4 F3 F1      [2]       [14, 16]    []          [17]      [18]
Beq R1 R2 8         [2]       [3, 3]      []          []        [18]
Mult.d F5 F4 F2     [5]       [18, 37]    []          [38]      [39]
Add R0 R1 R2        [5]       [6, 6]      []          [7]       [39]
```

Final state (identical for all widths): `R1 = R2 = 12`, `R3 = R4 = 0`, `F1 = 2.0`, `F2 = 3.5`, `F3 = 3.5`, `F4 = 1.5`, `F5 = 5.25`, `Mem[12] = 2.0`, `Mem[20] = 3.5`.

3.4 Sample report test case #01 (comparison)

This is the first test case of the sample report, run with my simulator.

Configuration of the Architecture

```
               # of rs         Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4               1                               1
FP adder        4               3                               1
FP multiplier   4               5                               1
Load/store unit 4               1               6               1
ROB entries = 64
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0
```

Issue width 1

```
                    ISSUE     EXE         MEM         WB        COMMIT
Ld F1 0(R1)         [1]       [2, 2]      [3, 8]      [9]       [10]
Mult.d F2 F1 F0     [2]       [10, 14]    []          [15]      [16]
Add.d F3 F3 F2      [3]       [16, 18]    []          [19]      [20]
Mult.d F2 F1 F1     [4]       [11, 15]    []          [16]      [21]
Add.d F4 F2 F4      [5]       [17, 19]    []          [20]      [22]
Addi R1 R1 8        [6]       [7, 7]      []          [8]       [23]
Ld F1 0(R1)         [7]       [9, 9]      [10, 15]    [17]      [24]
Mult.d F2 F1 F0     [8]       [18, 22]    []          [23]      [25]
Add.d F3 F3 F2      [9]       [24, 26]    []          [27]      [28]
Mult.d F2 F1 F1     [10]      [19, 23]    []          [24]      [29]
Add.d F4 F2 F4      [11]      [25, 27]    []          [28]      [30]
Addi R1 R1 8        [12]      [13, 13]    []          [14]      [31]
Ld F1 0(R1)         [13]      [15, 15]    [16, 21]    [22]      [32]
Mult.d F2 F1 F0     [14]      [23, 27]    []          [29]      [33]
Add.d F3 F3 F2      [17]      [30, 32]    []          [33]      [34]
Mult.d F2 F1 F1     [18]      [24, 28]    []          [30]      [35]
Add.d F4 F2 F4      [19]      [31, 33]    []          [34]      [36]
```

Issue width 2

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[2]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[20]
Add.d F4 F2 F4	[3]	[17, 19]	[]	[20]	[21]
Addi R1 R1 8	[3]	[4, 4]	[]	[5]	[21]
Ld F1 0(R1)	[4]	[6, 6]	[9, 14]	[15]	[22]
Mult.d F2 F1 F0	[4]	[16, 20]	[]	[21]	[22]
Add.d F3 F3 F2	[5]	[22, 24]	[]	[25]	[26]
Mult.d F2 F1 F1	[5]	[17, 21]	[]	[22]	[26]
Add.d F4 F2 F4	[6]	[23, 25]	[]	[26]	[27]
Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[27]
Ld F1 0(R1)	[7]	[9, 9]	[15, 20]	[21]	[28]
Mult.d F2 F1 F0	[11]	[22, 26]	[]	[27]	[28]
Add.d F3 F3 F2	[17]	[28, 30]	[]	[31]	[32]
Mult.d F2 F1 F1	[17]	[23, 27]	[]	[28]	[32]
Add.d F4 F2 F4	[18]	[29, 31]	[]	[32]	[33]

Issue width 4

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[1]	[11, 15]	[]	[16]	[20]
Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[21]
Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[21]
Ld F1 0(R1)	[2]	[5, 5]	[9, 14]	[15]	[21]
Mult.d F2 F1 F0	[2]	[16, 20]	[]	[21]	[22]
Add.d F3 F3 F2	[3]	[22, 24]	[]	[25]	[26]
Mult.d F2 F1 F1	[3]	[17, 21]	[]	[22]	[26]
Add.d F4 F2 F4	[3]	[23, 25]	[]	[26]	[27]
Addi R1 R1 8	[3]	[5, 5]	[]	[6]	[27]
Ld F1 0(R1)	[4]	[7, 7]	[15, 20]	[21]	[27]
Mult.d F2 F1 F0	[11]	[22, 26]	[]	[27]	[28]
Add.d F3 F3 F2	[17]	[28, 30]	[]	[31]	[32]
Mult.d F2 F1 F1	[17]	[23, 27]	[]	[28]	[32]
Add.d F4 F2 F4	[18]	[29, 31]	[]	[32]	[33]

Compared with Figures 1, 2 and 4 of the sample report the tables are the same except where the three modelling differences of Section 2.9 apply. With issue width 1: the first `Mult.d F2 F1 F1` is broadcast in cycle 16 and the second `Ld F1` in 17 (oldest first; the sample has 17 and 16), the second `Mult.d F2 F1 F0` and `Mult.d F2 F1 F1` therefore execute one cycle later ([18, 22] and [19, 23]), and the last three instructions issue one cycle later (17, 18, 19 instead of 16, 17, 18) because a freed reservation station is reused in the next cycle. With issue width 2 only the issue cycles of the instructions that wait for a reservation station differ (11, 17, 17, 18 instead of 10, 16, 16, 17); every EX, MEM, WB and COMMIT cycle is identical to Figure 2. With issue width 4 the second `Addi R1 R1 8` executes in cycle 5 instead of 4 because it waits for the broadcast of the first `Addi` (cycle 4) like every other instruction, (so its WB moves from 5 to 6) and the third `Ld F1` that depends on it computes its address in cycle 7 instead of 6; its memory access still starts in cycle 15, and apart from the reservation-station issue cycles everything else is the same as in Figure 4.